

Chapter 16 — Code and numerics

The audience's daily work

This is the chapter closest to what you actually do on a Tuesday, and the one where the tool is weakest at exactly the thing you need most.



Inherited code

A script from a student who graduated, no comments, and a result you have to defend.

- Ask for an explanation **before** asking for a change. You are buying a map, not a repair.
- Check the explanation against the code's behaviour, not against its variable names. Names lie; behaviour does not.
- The parts it explains confidently and wrongly are the parts to read yourself first.

The output of this step is a project instruction file entry, per Chapter 10 — so the next person does not pay the same cost.



A weakness worth showing you

Model performance on MATLAB is materially worse than on Python.

- You will feel this on your own code, so it is better heard here than discovered alone.
- Shown live, not asserted. Same task, both languages, side by side, on the day.

`TODO(capture)` — instructor records the demonstration in advance: several tasks, and more than one run of each, with model ID and date. One task at one run per language is the design Chapter 4 spent forty minutes forbidding, and this room will remember that. Report what was measured rather than asserting "materially worse" in advance — if the gap is smaller than expected, that is the finding.

Keep the observation and the explanation apart. That the gap exists is demonstrable in the room. That it is caused by how much MATLAB appears in training corpora is a hypothesis — plausible, widely repeated, and not established on this slide. `TODO(cite)` — search: MATLAB code generation LLM evaluation benchmark .



Why admitting that helps

- A limitation that costs the instructor something is the cheapest credibility available.
- It also gives you a decision rule: prototype in whichever language the tool is strong in, then port deliberately, or stay in MATLAB and budget more review.
- And it is the Chapter 4 lesson again — measure it on your own tasks rather than believing either me or the vendor.



Translation between MATLAB and Python

- Indexing base, array copy semantics, and default numeric types are where translations break silently.
- A translated script that runs is not a translated script that agrees. **Compare outputs on a case with a known answer** before trusting it on one without.
- Translation is a good use of the tool precisely because the check is cheap and mechanical.



Vectorisation and performance

- Ask for the vectorised form, then time both. The rewrite is often right and occasionally slower.
- Profile before optimising. Chapter 7's lesson generalises: find the binding constraint before working on anything.
- A vectorised rewrite **can** change results — floating-point addition is not associative, and vectorising reorders the reductions. The change is not confined to edge cases; it is wherever the summation order moved. Compare against the original, not against intuition.



Tests for numerical code

- Tolerance is a choice you must justify. A test passing at $1e-6$ and failing at $1e-9$ is telling you something about your method, not about the test.
- Prefer checks the physics gives you free: conservation, closure, symmetry, known limits, convergence under refinement.
- A test the tool wrote and you did not read is not a test. It is a second thing to debug.

State the check before you run the work. It is the same demand Chapter 4 made of a prompt and Chapter 14 made of a citation, arriving here for a tolerance.



Units, signs and coordinate systems

- The failure class most likely to survive review, because the code runs and the plot looks fine.
- Annotate them in comments, at the point of definition — not in a document nobody opens.
- Put them in the project instruction file so every future session inherits them.

Review cannot catch a sign error that nobody wrote down. That is true of human reviewers and it is true of this tool.



Exercise

1. Take an inherited script. Get an explanation and check it against behaviour.
2. Write one test with a tolerance you can justify out loud.
3. Add the units and sign conventions to your project instruction file.

